

PRNG

telemetry 백도어 복원, 다음 출력 예측과 구현 최적화

2026년 7월

결과 요약

 $r_3 = 0x2443c8daf1a9d52b09$

복원량	값
비밀 스칼라 d	$0x1c3cdd6b221806db0a7b28$
r_0 경로에서 얻는 s_2	$0x638d9d631ab436da51e640$
r_1 경로에서 직접 얻는 s_3	$0x948173253ad6d120a3f562$
다음 출력 r_3	$0x2443c8daf1a9d52b09$

평가 항목	근거 절
출력 값 예측 정확성 (20점)	3절
백도어 비밀값 복원 여부 · 정확성 (20점)	2절
분석 과정 문서화 (30점)	전체 절, 특히 2-5절
구현 기법의 완성도 · 효율성 (30점)	4, 6, 7절

1 공격 개요

문제의 생성식은 내부 상태 s_i 에 대해

$$s_{i+1} = X(s_i P), \quad r_i = \text{TMSB}(X(s_{i+1} Q))$$

이다. 공개 출력은 88비트 x 좌표의 하위 16비트를 버린 72비트다. telemetry의 여섯 행은 같은 비밀 스칼라 d 에 대한 affine residue의 하위 20비트를 버린 값이다. 풀이의 전체 흐름은 다음과 같다.

1. telemetry 생성식을 추론하고 여섯 행으로 d 를 유일하게 복원한다.
2. 독립 점 등식 $P = dQ$ 로 추론한 식과 d 를 검증한다.
3. 공개 출력 하나에 하위 16비트를 붙여 가능한 곡선점을 lift한다.
4. 숨은 관계로 다음 내부 상태를 얻고 나머지 공개 출력을 재생해 잘못된 후보를 제거한다.
5. 유일한 상태에서 r_3 를 계산한다.

상태 표기. r_0 을 lift한 점은 $s_1 Q$ 이고 $X(d(s_1 Q)) = X(s_1 P) = s_2$ 다. 같은 공격을 r_1 에 적용하면 직접 복원되는 값은 s_3 다. 제출 답과 모든 검증에서 이 index를 구분했다.

2 telemetry에서 비밀 스칼라 복원

2.1 누설식 추론과 검증

부분군 위수

$$n = 0x2b674bdfdf6fc4ba4ba751d, \quad B = 2^{20}$$

에 대해 각 행의 scale, offset, summary를 다음의 가장 단순한 affine-residue 모형으로 해석했다.

$$u_i = (\text{scale}_i d + \text{offset}_i) \bmod n,$$

$$\text{summary}_i = \lfloor u_i / B \rfloor.$$

문제는 정확한 식을 주지 않으므로, 이는 검증이 필요한 가설로 다룬다. 여섯 scale_i 가 모두 n 과 서로소이므로 역원이 존재한다. 첫 행에 대해

$$(\text{scale}_0 d + \text{offset}_0) \bmod n = \text{summary}_0 B + \ell, \quad 0 \leq \ell < B$$

이고, $v = \text{scale}_0^{-1} \bmod n$, $\ell = 0$ 일 때의 후보를 d_0 라 하면

$$d(\ell) = d_0 + \ell v \bmod n$$

이다. 이 식을 둘째 행에 넣으면 미지수 하나 ℓ 에 대한 modular interval 문제가 된다.

2.2 floor_sum을 이용한 비열거 복원

$$a = \text{scale}_1 v \bmod n, \quad c = \text{scale}_1 d_0 + \text{offset}_1 \bmod n$$

이라 놓으면 둘째 행은

$$(a\ell + c) \bmod n \in [L, U)$$

가 된다. 여기서 $L = \text{summary}_1 B$, $U = \min((\text{summary}_1 + 1)B, n)$ 다.

Euclidean recurrence로

$$F(k, m, a, b) = \sum_{i=0}^{k-1} \left\lfloor \frac{ai + b}{m} \right\rfloor$$

를 계산하면 다음 식으로 임의의 ℓ 구간에 조건을 만족하는 원소가 몇 개인지 $O(\log n)$ 에 셀 수 있다.

$$\#\{i < k : (ai + b) \bmod m < y\} = k - \{F(k, m, a, b + m - y) - F(k, m, a, b)\}.$$

해가 있는 구간만 이분하여 유일한 $\ell = 0x1f051$ 을 찾았다. 복원한

$$d = 0x1c3cdd6b221806db0a7b28$$

는 여섯 행 모두에서

$$((\text{scale}_i d + \text{offset}_i) \bmod n) \gg 20 = \text{summary}_i$$

를 만족한다. 또한 전체 좌표를 사용한 독립 검증 $P = dQ$ 도 성립해, 추정한 d 를 telemetry 외부에서도 확인했다.

기존 2^{20} 전수조사는 warm 반복 중앙값 1,453.475ms였고, floor_sum 경로는 0.750ms였다. 이 인스턴스에서는 약 1,938배 차이이며, 복잡도는 $O(B)$ modular operation에서 $O(\log B \log n)$ 으로 줄었다.

3 출력 lift와 r_3 예측

공개 출력 r_j 마다

$$x = (r_j \ll 16) \mid \ell, \quad 0 \leq \ell < 2^{16}$$

를 만들고 $x < p$ 이며

$$y^2 = x^3 + ax + b \pmod{p}$$

가 해를 갖는 후보만 남긴다. $+y$ 와 $-y$ 는 서로 반대점이고 $X(dR) = X(-dR)$ 이므로 한 부호만 검사한다.

기본 경로는 r_0 을 lift한 $R = \pm s_1 Q$ 에서

$$s_2 = X(dR)$$

를 얻는다. 이 상태로 r_1 을 재생해 후보를 거르고, $s_3 = X(s_2 P)$ 에서 r_2 도 재생한다. 유일한 생존 후보에 대해

$$s_4 = X(s_3 P), \quad r_3 = \text{TMSB}(X(s_4 Q))$$

를 계산하면 제출 답을 얻는다.

최종 native 탐색은 같은 원리를 r_1, r_2 관측 window에 적용한다. r_1 의 lift $T = \pm s_2 Q$ 에서 $X(dT) = s_3$ 를 얻고 r_2 로 필터한 뒤 r_3 를 계산한다. 정답 low가 0x5338에서 0x3cea로 바뀌어 순차 prefix가 21,305개에서 15,595개로 26.8% 줄었다. 40개 adjacent AB/BA pair에서 이 변경의 paired median은 $1.3428\times$, bootstrap 95% CI는 $1.3336 \dots 1.3510$ 이며 stationarity gate를 통과했다.

4 최종 구현

4.1 고정폭 체와 타원곡선 연산

최고 성능 C++20 경로는 88비트 소수체를 16바이트 2-limb Montgomery residue로 표현한다. BMI2/ADX target에서는 `_mulx_u64`와 `carry/borrow intrinsic`을 쓰고, 그 외에는 `portable unsigned __int128` 경로로 fallback한다. Jacobian point는 48바이트 POD다.

임의점 d 배는 원곡선을 $a = -3$ 인 동형 곡선으로 옮겨 Hamburg co-Z x -only ladder를 사용한다. exceptional denominator는 width-2 NAF로 fail-safe 처리한다. 정상 경로에는 y 가 필요 없으므로 모든 lift에서 제곱근을 계산하지 않고 Montgomery residue에 직접 적용한 hybrid 128/64비트 Euclidean Jacobi symbol로 제곱잉여를 먼저 판정한다. 실제 제곱근은 NAF fallback이 필요한 예외에서만 지연 계산한다.

반복되는 고정점 Q 곱에는

$$\text{table}[i][j] = j \cdot 2^{8i} Q \quad (0 \leq i < 11, 0 \leq j < 256)$$

인 byte-comb를 사용한다. table을 한 번 batch-normalize해 90,112바이트 affine 배열로 저장하므로 이후 고정점 곱은 최대 11회의 mixed addition으로 끝난다.

4.2 cofactor-5 부분군 필터

이 곡선은 $\#E(\mathbb{F}_p) = 5n$, $\text{ord}(Q) = n$ 이다. 따라서 curve-valid lift의 약 1/5만 올바른 부분군에 있다. 모든 후보에 비싼 Hamburg 곱을 하기 전에 x -only Frobenius-Tate trace로 $[n]T = O$ 와 동치인 부분군 membership을 판정한다.

$p \bmod 5 = 4$ 인 dual embedding degree 2를 $\mathbb{F}_{p^2}$ Frobenius -1 eigenspace의 order-5 점에 특수화했다. Miller 값에서

$$W = f(T)^p / f(T), \quad \tau = W + W^{-1}$$

를 만들면 y 가 소거되어 x 만으로 계산할 수 있다. $E = (p+1)/5 = 20H$ 라 두고 Lucas recurrence로 L_H 를 계산한 뒤 $\mu_{20}/\{z \sim z^{-1}\}$ 의 11개 trace와 비교한다. 고정 PRAC schedule은 115 byte, $118M + 6S = 124$ products로, binary Lucas의 $85M + 85S = 170$ 보다 27.1% 적다.

정답 prefix의 curve-valid lift 7,713개에서 직접 $[n]T = O$ 인 후보 1,547개와 trace 판정 1,547개가 전부 일치했다. 이 필터는 Hamburg 호출의 79.94%를 제거한다.

4.3 shifted-square trace와 batch inversion

expanded trace의 공통 인수를 제거하고 $z = (x - \alpha)^{-1}$ 로 바꾸면 처음에는 5차 reciprocal polynomial을 Horner $5M$ 으로 계산할 수 있다. 다섯 계수는 다시 정확히

$$\begin{aligned} r &= \lambda z, \\ \tau &= 2 + r((r+h)^2 + k)^2 \end{aligned}$$

로 인수분해된다. λ^5 는 최고차항 계수다. 차수 5를 만드는 데 field product 두 개만으로는 부족하므로 $2S + 1M$ 은 multiplicative-degree 하한을 달성한다.

binary-GCD 역원이 마지막에 Montgomery domain으로 들어갈 때 λR^2 를 곱해 $r = \lambda z$ 를 추가 product 없이 얻었다. 또한 Montgomery batch inversion의 첫 forward와 마지막 두 reverse endpoint product를 생략했다. 활성 원소가 m 개일 때 normalization+trace 비용은

$$8m + I \longrightarrow 6m - 3 + I$$

로 줄고, PRAC까지 포함한 block membership은

$$130m - 3 + I$$

이다. 3-thread 이상 scalar 경로는 후보당 $127M/S + I$ 다. I 는 field inversion이며 block에서는 하나를 공유한다.

4.4 연속 cubic recurrence와 병렬 정책

연속 low-bit 후보는 고정 step h 에 대해 $f(x) = x^3 + ax + b$ 를 계산한다. 각 block 첫 원소에서만 직접 평가하고

$$\begin{aligned}\Delta f &= 3x^2h + 3xh^2 + h^3 + ah, \\ \Delta^2 f &= 6xh^2 + 6h^3, \\ \Delta^3 f &= 6h^3\end{aligned}$$

를 field addition으로 갱신한다. recurrence가 곡선 우변을 공급하므로 Hamburg용 x^2 는 Jacobi 직후가 아니라 부분군 필터를 통과한 후보만 계산한다. 정답 prefix에서 field square 6,166회를 제거한다.

작업은 atomic counter로 연속 candidate block을 배분한다. 검증된 adaptive 정책은 1 thread에서 block 64, 2 threads에서 block 32, 3 threads 이상에서 scalar chunk 64다. 2-thread 정책만 별도의 고정 CPU 40-pair 측정에서 승격됐으므로 다른 thread 수로 성능을 보간하지 않았다. AVX2에는 직접적인 packed $64 \times 64 \rightarrow 128$ 정수 곱이 없어 radix 분해와 lane compaction 비용이 커지는 까닭에 기본 경로로 채택하지 않았다.

5 정확성 검증

성능 후보는 timing 전에 다음 oracle을 통과해야 한다.

- canonical arithmetic와 2,000개 field vector, 64개 경계 pair
- affine reference와 256개 point/scalar vector
- 실제 lift 128개에서 Hamburg/NAF 및 scalar/batched subgroup 대조
- Sage 무작위 200점과 실제 prefix 전체에서 trace와 직접 $[n]T = O$ 비교
- expanded, reciprocal, Horner, shifted-square 식의 경계·무작위·공개 prefix 3 · 65,536개 cross-product identity
- scaled binary inverse와 일반 inverse oracle, endpoint-elided batch의 경계 크기·분모 0 fail-closed 검사
- $P = dQ$, d , 상태 label, 정답 low bits와 r_3 known answer

최종 native --self-test --json은 다음을 출력했다.

```
{"self_test":true,
"field_vectors":2000,
"field_boundary_pairs":64,
"point_vectors":256,
"hamburg_lift_vectors":128,
"subgroup_lift_vectors":128}
```

실제 공격 결과도 다음 known-answer 필드를 모두 확인한다.

```
d=0x1c3cdd6b221806db0a7b28
state_label=s3
state=0x948173253ad6d120a3f562
lift_low_bits=15594
r3=0x2443c8daf1a9d52b09
p_equals_dq=true
```

6 반복 측정과 해석 한계

측정 환경은 AMD EPYC 7B12 VM, 8 logical CPU, Python 3.11.2, G++ 12.2.0이었다. 큰 단계 비교는 1회 warm-up 뒤 완전한 공격을 5회 교차 실행했다. 작은 후보는 adjacent AB/BA 40쌍, bootstrap 95% CI, 실행 순서 stratum과 시간 block stationarity를 함께 검사했다. 매 timing 전에 정답과 실제 thread/schedule/backend metadata를 확인했다.

역사적 구현	중앙값	Python 대비	paired
기존 Python	14.298741s	1.00×	1.00×
최적화 Python int	3.272242s	4.37×	4.41×
최적화 Python gmpy2	3.000356s	4.77×	4.61×
C++/GMP 1 thread	1.873220s	7.63×	7.44×
C++/GMP/OpenMP 8 threads	0.447919s	31.92×	30.75×

위 표는 언어/backend 전환 당시의 동일 campaign 결과이며, 후속 Jacobi, 부분군 trace/PRAC, shifted-square와 adaptive 정책까지 포함한 현재 source의 절대 시간표는 아니다. 각 배수는 같은 host 부하 조건에서 측정한 값만으로 계산했다.

안정적으로 stationarity gate를 통과한 구성요소 효과는 r_1, r_2 window 1.3428×, Hamburg 1.1716×, sqrt 대신 Jacobi 1.0819×, 2-thread adaptive block 1.2121×였다.

shifted-square/Horner는 paired 1.0077× (CI 1.0043...1.0149)였지만 2% 승격 문턱과 절대-time stationarity를 통과하지 못했다. 다만 분리한 normalization+trace cycle은 활성 원소 32/128/256개에서 1.269/1.304/1.313×였고, $8m + I \rightarrow 6m - 3 + I$ 의 정확한 operation 감소와 Sage/C++ 등가 검증이 성립하므로 기본값으로 채택했다.

cubic recurrence와 post-subgroup x^2 지연도 wall-clock stationarity를 통과하지 못했다. 이들은 각각 direct cubic multiplication과 실제로 버려지는 후보의 square를 제거하는 진부분집합 최적화이며, 전수 등가 검사와 기존 경로를 재현하는 ablation macro를 함께 남겼다. 이 보고서에서 “stationarity 실패”는 해당 공유 VM 측정에서 작은 성능 차이를 통계적으로 확정하지 못했다는 뜻이며, 정확성은 앞선 정확성 검증 절에서 별도로 확인했다.

7 복잡도와 메모리

- telemetry 복원: $O(\log B \log n)$, $B = 2^{20}$.
- 상태 복원: 최악 2^{16} 개의 x 후보에 Jacobi와 부분군 판정을 수행하고, 생존 후보에 고정 d 곱과 공개 출력 검증을 수행하므로 $O(2^{16} \log n)$.
- 다음 출력: 상태가 정해진 뒤 상수 횟수의 scalar multiplication.
- 고정 메모리: 90,112바이트 affine Q table. 그 밖의 탐색 상태는 worker마다 상수 크기이며 전체 2^{16} 후보를 저장하지 않는다.

8 재현

다음처럼 최종 native source를 빌드한다.

```
cd src
g++ -O3 -DNDEBUG -march=native -std=c++20 -fopenmp \
    deep_native.cpp -o /tmp/deep_native

/tmp/deep_native --self-test --json
/tmp/deep_native --threads 1 --schedule adaptive --json
```

PRAC schedule과 부분군 수학의 독립 감사는 다음 명령으로 재현한다.

```
python3 generate_prac_schedule.py --json
sage -python audit_subgroup.py --json
sage -python audit_subgroup.py --samples 0 --full-prefix --json
```

PRAC 생성기는 source_match=true, length=115, field_products=124를 출력해야 한다. Sage full-prefix 감사는 prefix_valid_lifts=7713, prefix_members=1547을 출력해야 한다.

9 참고 자료

1. NIST, SP 800-90 Rev. 1 (withdrawn). <https://csrc.nist.gov/pubs/sp/800/90/r1/final>

-
2. D. Shumow and N. Ferguson, **On the Possibility of a Back Door in the NIST SP800-90 Dual EC PRNG.**
<https://rump2007.cr.yp.to/15-shumow.pdf>
 3. AtCoder Library, floor_sum documentation and implementation.
https://atcoder.github.io/ac-library/production/document_en/math.html
 4. Explicit-Formulas Database, short-Weierstrass Jacobian formulas.
<https://www.hyperelliptic.org/EFD/g1p/auto-shortw-jacobian.html>
 5. M. Hamburg, **Faster Montgomery and double-add ladders for short Weierstrass curves**, ePrint 2020/437. <https://eprint.iacr.org/2020/437>
 6. N. Möller, **Efficient computation of the Jacobi symbol.** <https://arxiv.org/abs/1907.07795>
 7. A. Koshelev, **Subgroup membership testing on elliptic curves via the Tate pairing**, ePrint 2022/037 및 출판 correction. <https://eprint.iacr.org/2022/037.pdf>
 8. A. Enge, **Bilinear pairings on elliptic curves.** <https://arxiv.org/abs/1301.5520>
 9. P. L. Montgomery, **Evaluating recurrences of form $X_{m+n} = f(X_m, X_n, X_{m-n})$ via Lucas chains.**
<https://cr.yp.to/bib/1992/montgomery-lucas.pdf>
 10. P. L. Montgomery, **Modular Multiplication Without Trial Division.**
<https://doi.org/10.1090/S0025-5718-1985-0777282-X>
 11. T. Mytkowicz et al., **Producing Wrong Data Without Doing Anything Obviously Wrong!**
<https://sape.inf.usi.ch/publications/asplos09.html>